



《人工智能综合设计》— Web UI



人工智能综合设计 教研组



提纲



人工智能综合设计06 Web UI

- ☐ Web 服务器
- ☐ GET/POST 请求
- ☐ 搜索引擎 Web UI



Web 服务器

- 通过 HTTP 协议接收请求 (request)
- 分发 Web 页面 (response)



Web 服务器

- 严格地说，Web 服务器包括硬件+软件
- 本次课程中，限定讨论范围在**软件**层面
- （大家自己的电脑作为硬件）
- 即开发 Python 程序作为 Web 服务软件，在自己的电脑上运行 Web 服务，并通过浏览器查看效果



Web 服务器

- Web 服务软件：一直运行的程序，除非手动终止（或者出 bug）



Web 服务器

- Web 服务器接收请求 (request) 后:
 1. 在后台执行一定的功能, 如获取数据、计算倒列表等
 2. 返回对应的网页



Web 服务器

- Web 服务软件的开发，俗称**后端**开发
- 对应 html (js, css, ...) 页面的编写，俗称**前端**开发



提纲



人工智能综合设计06 Web UI

- □ Web 服务器
- □ GET/POST 请求
- □ 搜索引擎 Web UI

请求



- HTTP 协议中定义了多种请求方式
- GET
- POST
- ...



GET 请求

- 最常用的请求方式
- 如直接输入 URL 请求资源往往是 GET 请求
- <http://ai.ruc.edu.cn/>





GET 请求

- 能否为 url 增加 “参数”
- 比如高瓴人工智能学院教师首页的 url 前缀统一为
- <https://gsai.ruc.edu.cn/addons/teacher/index/info.html>
- 那如何识别为哪一位教师?

GET 请求



- https://gsai.ruc.edu.cn/addons/teacher/index/info.html?user_id=0&rucode=20130047&ln=cn



https://gsai.ruc.edu.cn/addons/teacher/index/info.html?user_id=0&rucode=201300478



中国人民大学高瓴人工智能学院
Gao Ling School of Artificial Intelligence, Renmin University of China

[首页](#) [教师中心](#) [EN](#)



文继荣

教授 (长聘教授)

教育经历

- 1996 ~ 1999, 中国科学院计算技术研究所 博士研究生
- 1994 ~ 1996, 中国人民大学信息学院 硕士研究生
- 1990 ~ 1994, 中国人民大学信息学院 本科生

工作经历



GET 请求

- 以 ? 开头，为 GET 请求增加“参数”
- 每一个参数为“参数名=值”的形式
- 不同参数之间用 & 分隔
- `info.html?user_id=0&rucode=20130047&ln=cn`

GET 请求



- 缺点:
- 长度限制 1024K
- URL 明文传输, 不安全



POST 请求

- 无长度限制
- 不使用 URL 明文传输
- 比如某 GET 请求，请求一次后，网页会存在浏览器缓存中，之后可以通过“回退”等操作，在服务器不知情的情况下再次访问
- 而 POST 请求，“回退”时会再次请求服务器，服务器对两次访问均知情



发送 GET/POST 请求

- HTML 代码中表单 (Form)

```
<form action="query" method="get">
  <div>
    <input type="text" name='key' placeholder="输入您想查询的关键词...">
  </div>
  <div>
    <button type="submit">Go!</button>
  </div>
</form>
```




提纲



人工智能综合设计06 Web UI

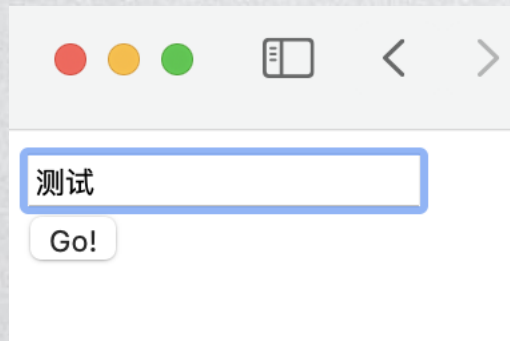
- ☐ Web 服务器
- ☐ GET/POST 请求
- ☐ 搜索引擎 Web UI



用户友好的搜索引擎

- 两个页面：查询页面接收关键词；结果页面返回搜索结果 url

查询页面



结果页面

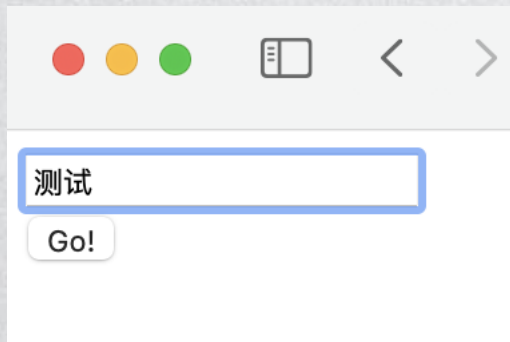




用户友好的搜索引擎

- 目标: **接入**已经写好的简单前后端程序
- 用户在查询框中输入**关键词**, 点击按钮, 返回包含搜索结果的网页。

localhost:12345/



localhost:12345/query?key=测试



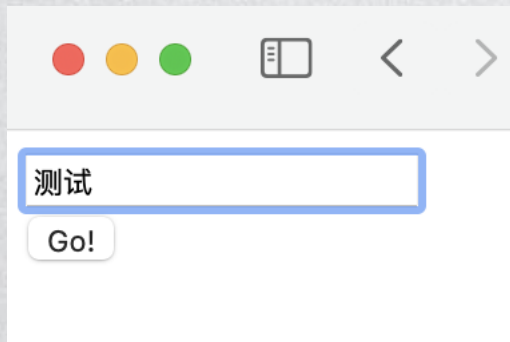
https://github.com/hyp1231/search_demo



用户友好的搜索引擎

- 网页前端是一个 form 表单，用户输入查询词之后，点击按钮，将表单作为一个 GET 请求发送到服务端

localhost:12345/

A mockup of a web browser window. The address bar shows 'localhost:12345/'. Below it is a search form with a text input field containing the text '测试' (Test) and a button labeled 'Go!'.



前端：form表单

- 复习：第一天 PPT

```
<form action="query" method="get">
  <div>
    <input type="text" name='key' placeholder="输入您想查询的关键词...">
  </div>
  <div>
    <button type="submit">Go!</button>
  </div>
</form>
```

A preview of the HTML form rendered in a browser window. The window has a title bar with red, yellow, and green window control buttons, a maximize button, and left/right navigation arrows. The form itself consists of a text input field with the placeholder text "输入您想查询的关键词..." and a "Go!" submit button below it.

输入您想查询的关键词...

Go!



用户友好的搜索引擎

- 服务端接收 GET 请求，调用搜索引擎程序，返回包含搜索结果的网页。

localhost:12345/query?key=测试





用户友好的搜索引擎

- 服务端：Flask (一个 Python 库，可以方便接入 Python 语言开发的搜索引擎程序)
- Flask 作用：接收 url 请求，返回网页
- 再次强调：Web 服务软件是一个一直运行的程序，除非手动终止 (或者出 bug)
- 在运行的时候，只有接收请求的时候，程序才会做出反馈

Flask

- 只用 Python 就可以在本地启动一个 Web Server!
- 安装: `pip install flask`





Flask

- 安装好 flask 后，就可以运行 search_demo 了
- 下载 search_demo.zip
- 解压
- 运行 main.py

```
|— main.py
|— templates
    |— index.html
    |— res.html
```



Flask

- 被 @app.route 注册的 python 函数, 在 Server 接收了对应的 request 后执行

```
from flask import Flask, render_template, request
```

```
app = Flask(__name__)
```

 实例化

```
@app.route('/')
```

 接收 url

```
def index():
```

```
    return render_template('index.html')
```

返回 .html 网页

Flask

- 如何根据关键词 (query) 动态生成搜索结果页面?

localhost:12345/query?key=测试





Flask

- 将代码中这部分替换成你的搜索引擎程序

```
@app.route('/query', methods = ['GET'])
def query():
    key = request.args.get('key')

    # Implement your search engine here.
    # Generate a list of search results.

    results = [
        {'title': '百度', 'url': 'https://www.baidu.com/'},
        {'title': '哔哩哔哩', 'url': 'https://www.bilibili.com/'},
        {'title': '知乎', 'url': 'https://www.zhihu.com/'},
    ]

    return render_template('res.html', key=key, results=results)
```


Flask

- key – 输入的查询词; results – 搜索引擎返回的查询结果列表

```
@app.route('/query', methods = ['GET'])
def query():
    key = request.args.get('key')

    # Implement your search engine here.
    # Generate a list of search results.

    results = [
        {'title': '百度', 'url': 'https://www.baidu.com/'},
        {'title': '哔哩哔哩', 'url': 'https://www.bilibili.com/'},
        {'title': '知乎', 'url': 'https://www.zhihu.com/'}
    ]

    return render_template('res.html', key=key, results=results)
```



拓展

- 简单的修改，可以增加“摘要”等额外的展示信息
- （参考 search_demo_abstract.zip）

main.py

```
results = [  
    {'title': '百度', 'url': 'https://www.baidu.com/', 'abstract': '百度，一个著名'},  
    {'title': '哔哩哔哩', 'url': 'https://www.bilibili.com/', 'abstract': 'bilibili'},  
    {'title': '知乎', 'url': 'https://www.zhihu.com/', 'abstract': '知识分享社区'}  
]
```

templates/res.html

```
<div>  
    {% for res in results %}  
        <p>  
            <a href="{{ res['url'] }}">{{ res['title'] }}</a>  
        </p>  
        <p>  
            {{ res['abstract'] }}  
        </p>  
        <br>  
    {% endfor %}  
</div>
```

演示效果

Keys: 测试

[百度](#)

百度，一个著名的搜索引擎

[哔哩哔哩](#)

bilibili，弹幕视频网站

[知乎](#)

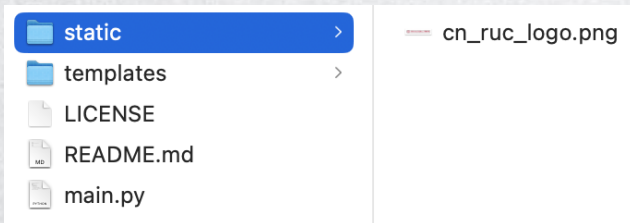
知识分享社区



拓展

- 插入图片

文件目录



templates/index.html

```
7 <body>
8 <div>
9 <form action="query" method="get">
10 <div>
11 <input type="text" name='key'
12     placeholder="输入您想查询的关键词...">
13 </div>
14 <div>
15 <button type="submit">Go!</button>
16 </div>
17 </form>
18 <div>
19 
22 </div>
23 </div>
```

演示效果

输入您想查询的关键词..

Go!



中国人民大学高瓴人工智能学院
Gaoling School of Artificial Intelligence, Renmin University of China